

Introduction to Unix/Linux

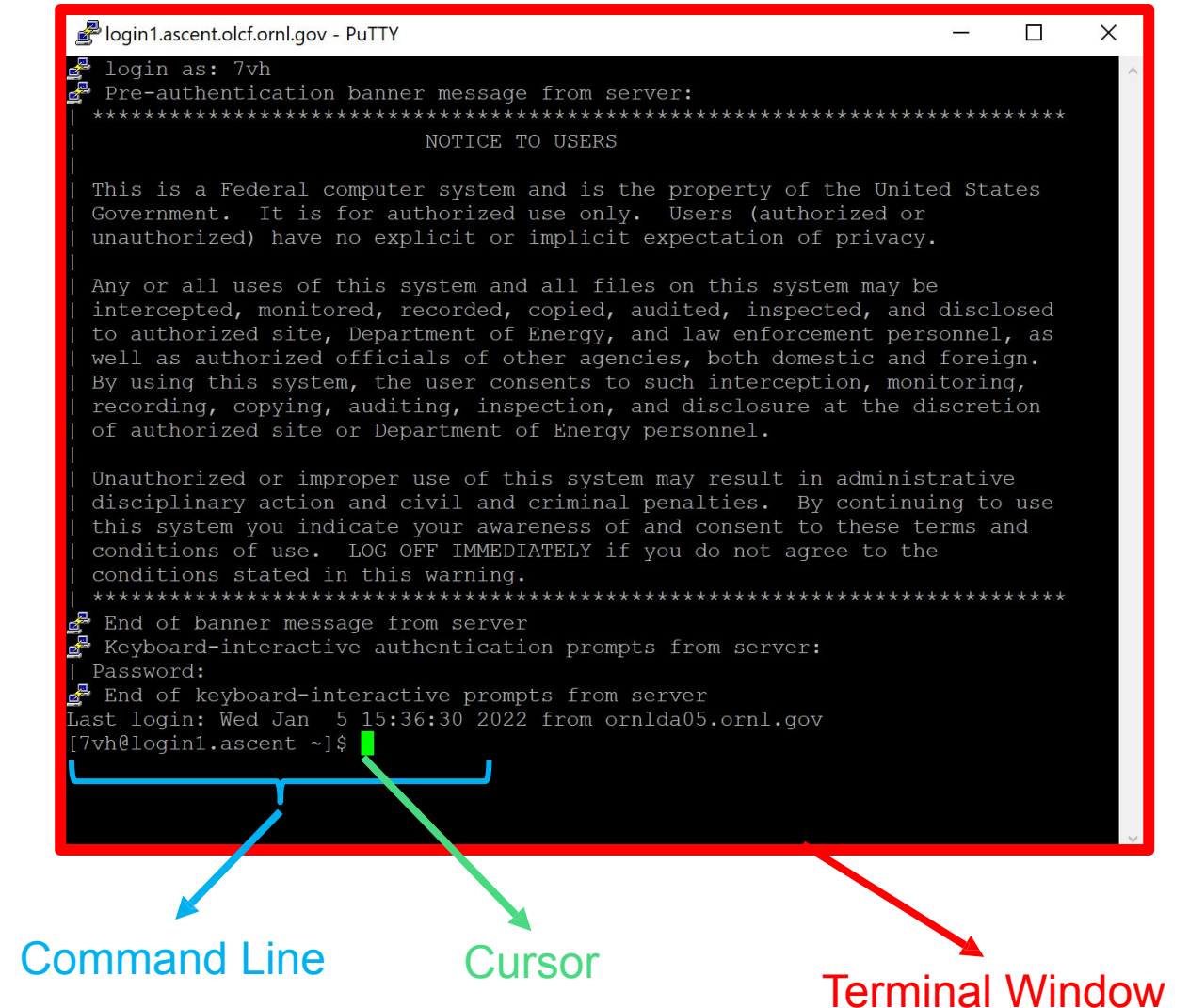
- This presentation introduces participants to basic Unix and Linux commands
 - The terminal will be used for experimentation and hands-on.
 - If you're using a macbook, just type terminal on your search to access your macbook terminal for this section.
 - Windows users can also type terminal for Powershell
- Windows users, install the following applications for terminal (**Recommended**)
 - Git Bash: <https://git-scm.com/install/windows>
 - MobaXterm: <https://mobaxterm.mobatek.net/download-home-edition.html>
 - Common commands to create, delete, edit, move, copy, etc., directories and files.

Overview

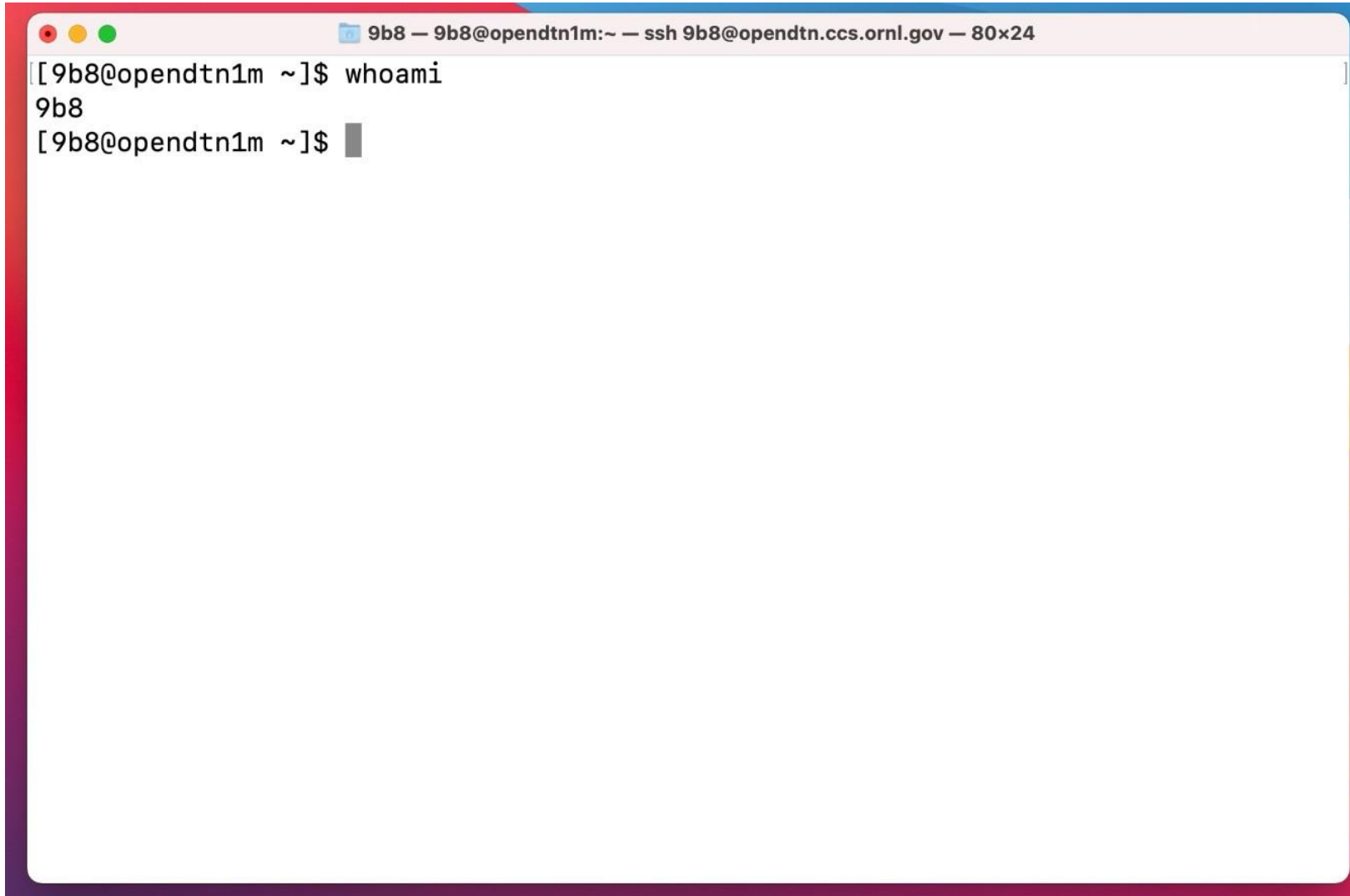
- Cover the basics
 - The terminal window, command line
 - The filesystem structure & how to navigate it
 - Common commands to create, delete, edit, move, copy, etc., directories and files.
- Google is your friend!

The Command Line

- Commands are typed as text on the command line and executed by pressing “enter”
- Many examples show the “\$” symbol from the command line before the actual command; you don’t need to type this symbol
- The command line often contains login and username info.
- Try typing **\$ whoami** and pressing enter



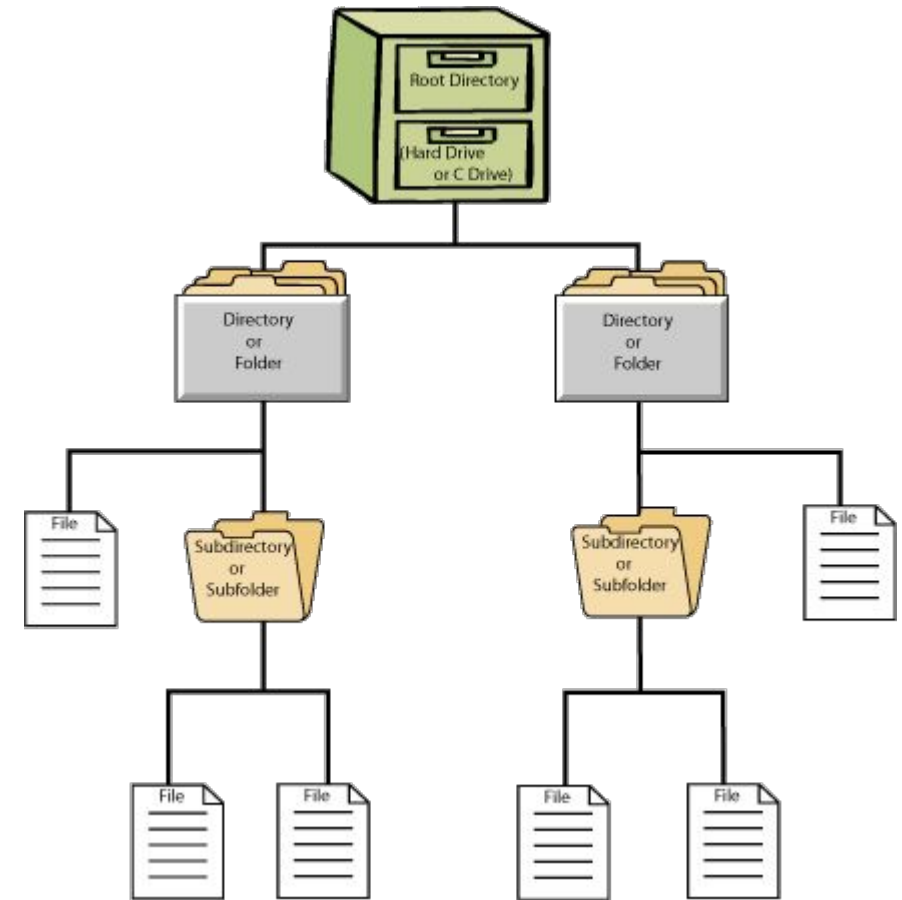
Who am I? (really though)

A terminal window with a title bar that reads "9b8 — 9b8@opendtn1m:~ — ssh 9b8@opendtn.ccs.ornl.gov — 80x24". The terminal shows the command "[9b8@opendtn1m ~]\$ whoami" being executed, followed by the output "9b8" on the next line. The prompt "[9b8@opendtn1m ~]\$" is shown again on the third line with a cursor. The terminal window has a red, yellow, and green window control bar at the top left.

```
[9b8@opendtn1m ~]$ whoami
9b8
[9b8@opendtn1m ~]$
```

The Filesystem 1

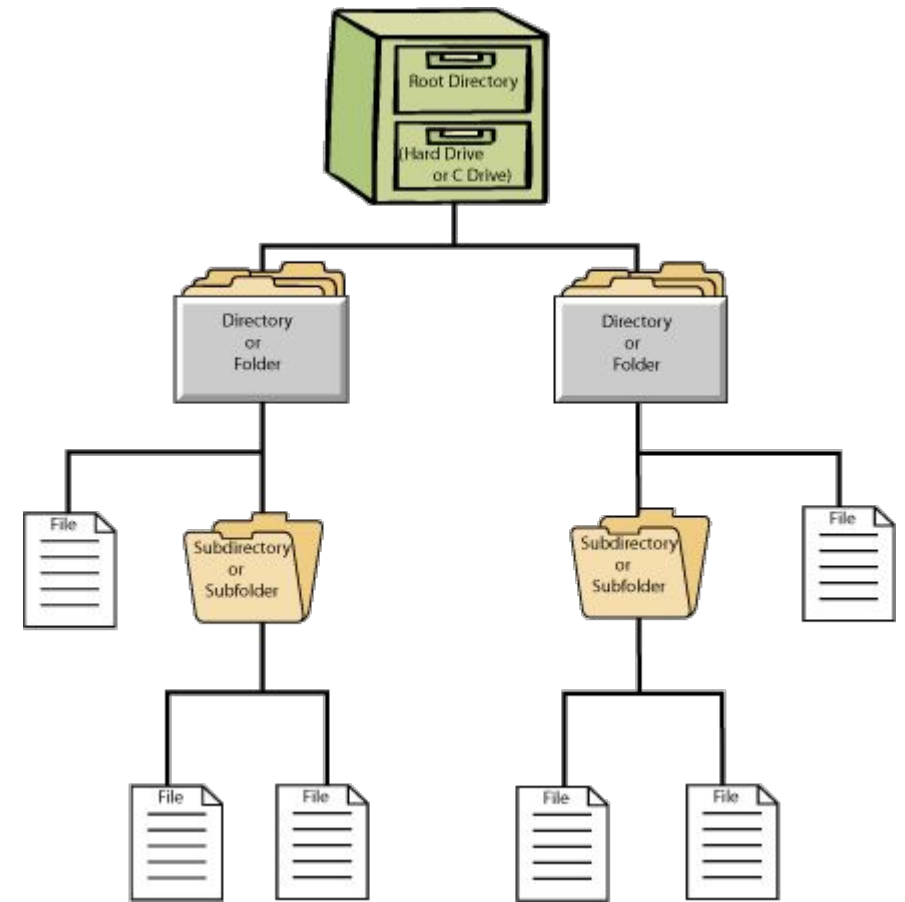
- **File** – A collection of data
 - Plain text, Input/output, a program (executable), an image, etc.
- **Directory** – A logical structure to help organize files (think “folder”)
- **Filesystem** – A collection of files and directories
- As a user, you land in your “home” directory when you log in, and can create directories and files there, or move elsewhere to do that.



The Filesystem 2

- Think of the filesystem as a tree. Instead of branches, there are “directories. Instead of leaves, there are “files”
- It starts at /, which is called the “root” directory
- The slash (/) is the directory separator; thus when we move into subdirectories it’s used to separate things (i.e. /c/home/your_username)

If you are ever “lost”, use the pwd command. It prints your location.



```
$ pwd  
/c/Users/maabe
```

Basic Commands

- Commands are usually abbreviations of words (or a series of words)
 - cp for “Copy”, rm for “Remove”, cd for “change directory”
- Commands tend to follow the following syntax:

```
[command] [option flags] [file/directory/object to act upon]
```
- Almost all commands take various options to control what they do.
- Some help is available via an online manual
 - The `man` command
 - Example: Want info about `ls`? Type the following:

```
man ls
```

Basic File/Directory Commands

Command	Description
pwd	Print w orking (i.e. current) d irectory
man	Display the M anual for a command
whoami	Display the current user's username
mkdir	Create a directory (Ma Ke DI Rectory)
rmdir	Delete a directory (Re Mov e DI Rectory)
cd	C hange (into a) d irectory
ls	L ist files
cp	C opy a file
mv	M ove a file (also used to rename a file)
rm	R emove (delete) a file
cat	Display the contents (con ca tenate) a (hopefully text) file
less	Display the contents of a text file in a viewing mode
find	F ind files by filename
grep	Search for text within files
diff	Identify d ifferences in the contents of files

“ls” and “cd” In Action 1

```
9b8 — 9b8@opendtn1m:~/foundational_hpc_skills/intro_to_unix/dir — ssh 9b8@opendtn.ccs.ornl.gov — 80x24
[[9b8@opendtn1m ~]$ man ls
[[9b8@opendtn1m ~]$ ls
foundational_hpc_skills  hands-on-with-summit  mp2.py
[[9b8@opendtn1m ~]$ cd foundational_hpc_skills/
[[9b8@opendtn1m foundational_hpc_skills]$ ls
intro_to_c      intro_to_ssh  intro_to_vim  README.md
intro_to_python intro_to_unix  presentations
[[9b8@opendtn1m foundational_hpc_skills]$ cd intro_to_unix/
[[9b8@opendtn1m intro_to_unix]$ ls
challenge  dir  README.md
[[9b8@opendtn1m intro_to_unix]$ cd dir/
[[9b8@opendtn1m dir]$ ls
file1  file1a  file1b  file2  file2a  file2b  foo
[[9b8@opendtn1m dir]$
```

Directories

Use `pwd`, `cd`, `mkdir`, and `rmdir` commands to navigate the filesystem and manipulate directories

```
$ pwd
/home/user1

$ mkdir dir1

$ ls
dir1

$ cd dir1

$ pwd
/home/user1/dir1

$ cd ..

$ rmdir dir1
```

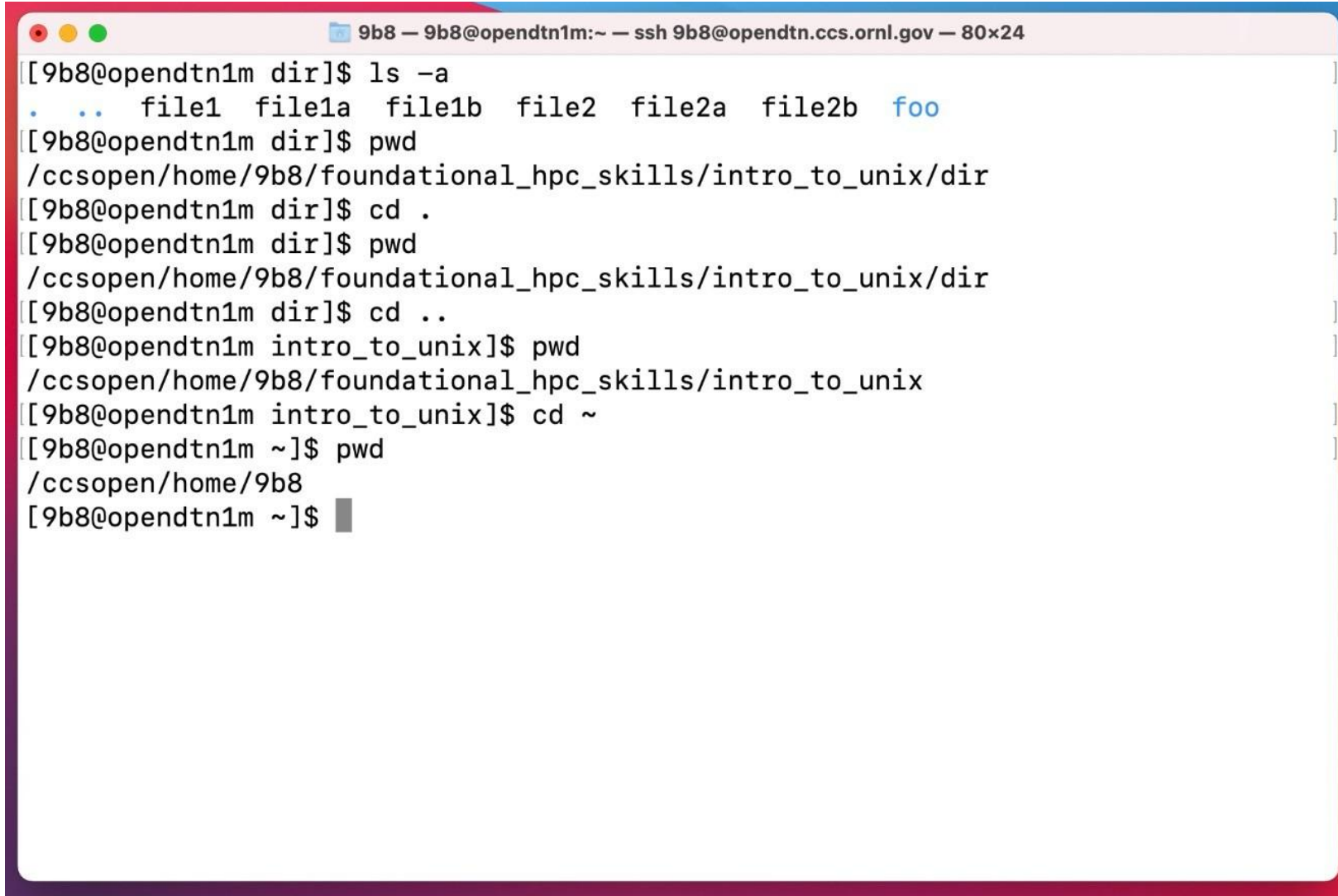
Typing just `cd` will always take you back to home no matter where you are.

Directories must be empty in order to delete them with `rmdir`

Special Directories

- Every directory contains two special directory entries: "." and ".."
- . is a reference to the current directory
- .. is a reference to the parent directory (so we can do things like `cd ..`)
- ~ can be a reference to home directories
 - ~/ is yours
 - ~user1/ is user1's
- You'll see how these are useful later

“ls” and “cd” In Action 2



```
9b8 — 9b8@opendtn1m:~ — ssh 9b8@opendtn.ccs.ornl.gov — 80x24
[9b8@opendtn1m dir]$ ls -a
.  ..  file1  file1a  file1b  file2  file2a  file2b  foo
[9b8@opendtn1m dir]$ pwd
/ccsopen/home/9b8/foundational_hpc_skills/intro_to_unix/dir
[9b8@opendtn1m dir]$ cd .
[9b8@opendtn1m dir]$ pwd
/ccsopen/home/9b8/foundational_hpc_skills/intro_to_unix/dir
[9b8@opendtn1m dir]$ cd ..
[9b8@opendtn1m intro_to_unix]$ pwd
/ccsopen/home/9b8/foundational_hpc_skills/intro_to_unix
[9b8@opendtn1m intro_to_unix]$ cd ~
[9b8@opendtn1m ~]$ pwd
/ccsopen/home/9b8
[9b8@opendtn1m ~]$
```

Wildcards

- When dealing with multiple files, it's nice to type a single command for all files at once vs. typing many separate commands for each file
- Wildcards help with this: They are generic characters that “fill in” for other characters
 - * means match zero or more character
 - ? Matches 1 character
 - Example follows (the `ls` command lists files in a directory, we'll worry about specific later)

Wildcards (example)

```
9b8 — 9b8@opendtn1m:~/foundational_hpc_skills/intro_to_unix/dir — ssh 9b8@opendtn.ccs.ornl.gov — 80x24
[9b8@opendtn1m dir]$ ls
file1 file1a file1b file2 file2a file2b foo
[9b8@opendtn1m dir]$ ls file1?
file1a file1b
[9b8@opendtn1m dir]$ ls file1*
file1 file1a file1b
[9b8@opendtn1m dir]$ ls file?a
file1a file2a
[9b8@opendtn1m dir]$
```

* means match zero or more characters

? Matches 1 character

```
$ ls
file1 file1b file2 file2a file2b file3 file3a file3b
file1a
$ ls file1?
file1a
file1b
$ ls file2*
file2 file2b
file2a
$ ls file?a
file1a file2a
```

More About “ls”

- Lists directory contents
- Helpful option: -l (shows many file attributes)

```
$ ls
filea  fileb

$ ls -l
total 0
-rw-r--r--  1 user1  group1  50 Jun 20 14:15 filea
-rw-r--r--  1 user1  group1   0 Jun 20 14:15 fileb
```



The diagram shows five curly brackets positioned below the output of the `ls -l` command. Each bracket spans a specific column of the output and is labeled with its corresponding field name: 'permissions' (under `-rw-r--r--`), 'owner' (under `1 user1`), 'group' (under `group1`), 'size' (under `50`), and 'name' (under `filea` and `fileb`).

Even More “ls” Info (Other Useful Options)

Option	Meaning
-l	Show one file per line (helpful in scripting)
-F	Show file types (directories, links, etc)
-a	Show all files (including hidden files)
-r	Reverse the order of the listing
-t	Sort files by timestamp
-d	List the (attributes of) the directory itself rather than listing its contents
...And many, many (many) more	

- You can combine options: `ls -altr` is the same as `ls -a -l -t -r`
(but more concise & w/less typing)

Manipulating Files 1

- The `rm` command is used to delete a file.
- The `mv` command is used to move and rename files
- There are multiple ways to create and view text files. In the challenge, we will look at various ways to use `cat` and `less` commands to do this.
- Utilities such as `less` and `cat` are intended only for text files. The system will not stop you from running them on a non-text file
 - If you do, you'll get a screenful of unintelligible characters
 - You might get a recognizable prompt (you might not)
 - There's no shame in closing that session's window & re-connecting

Manipulating Files 2

- The `cp` command is used to copy a file, and the `mv` command is used to move and rename files.

```
$ ls
dir1 filea fileb

$ cat filea
This is a file that
contains three
lines of text.

$ cp filea filea1

$ ls
dir1  filea filea1  fileb

$ mv filea1 filec

$ ls
dir1  filea  fileb  filec
```

Manipulating Files 3

- The `rm` command is used to delete a file. `cat` prints contents of a file to the screen. `less` displays the contents of a file in a viewing mode. (Press “q” to exit).

```
$ cat filec
This is a file that
contains three
lines of text.

$ less filec

$ mv filec dir1

$ ls dir1
filec

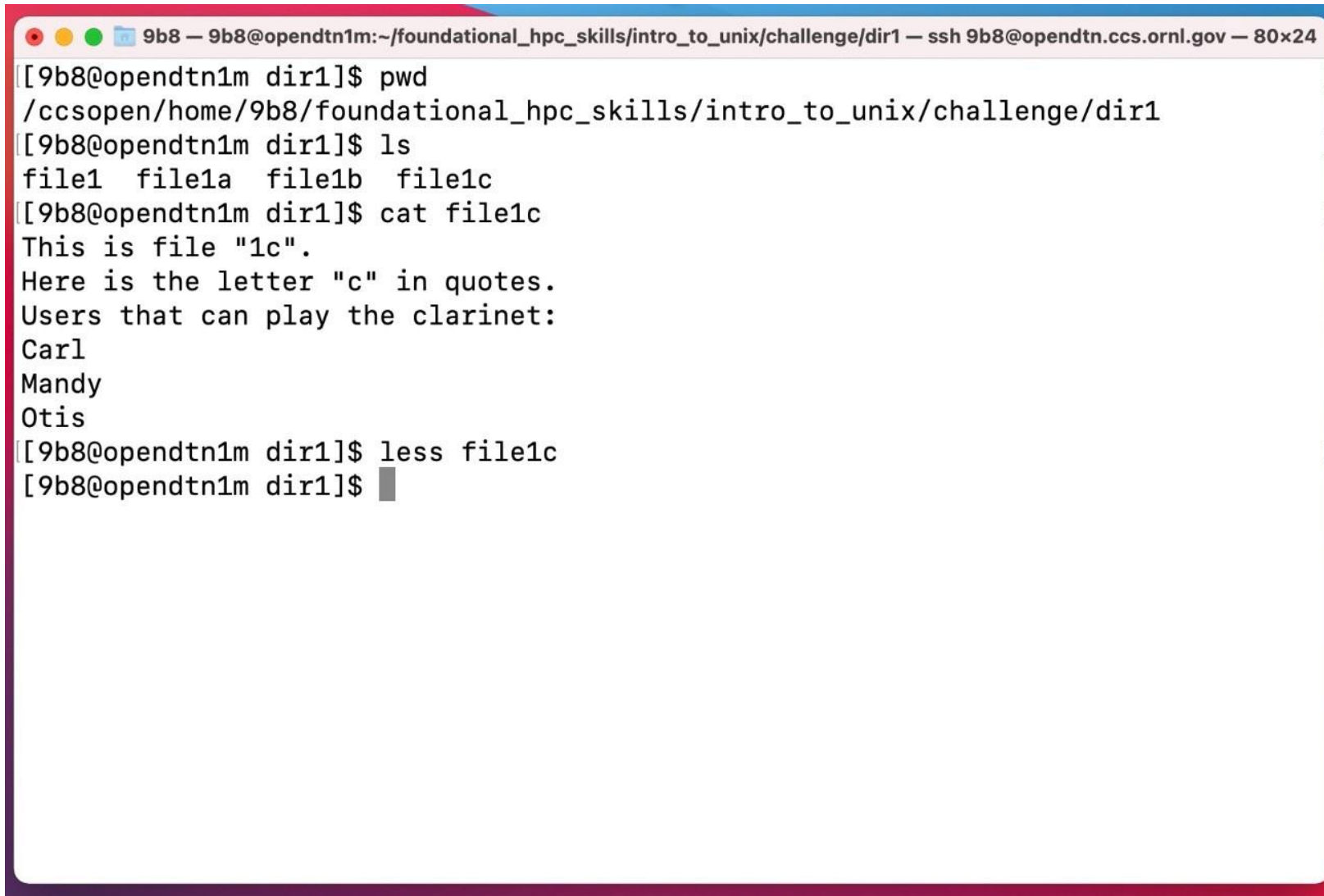
$ rm dir1/filec

$ ls
dir1      filea    fileb
```

Manipulating Files In Action

```
9b8 — 9b8@opendtn1m:~ — ssh 9b8@opendtn.ccs.ornl.gov — 80x24
foundational_hpc_skills hands-on-with-summit mp2.py
[9b8@opendtn1m ~]$ mkdir test_dir
[9b8@opendtn1m ~]$ ls
foundational_hpc_skills hands-on-with-summit mp2.py test_dir
[9b8@opendtn1m ~]$ cd test_dir/
[9b8@opendtn1m test_dir]$ ls
[9b8@opendtn1m test_dir]$ touch test_file.txt
[9b8@opendtn1m test_dir]$ ls
test_file.txt
[9b8@opendtn1m test_dir]$ cp test_file.txt test_file_2.txt
[9b8@opendtn1m test_dir]$ ls
test_file_2.txt test_file.txt
[9b8@opendtn1m test_dir]$ rm test_file_2.txt
[9b8@opendtn1m test_dir]$ ls
test_file.txt
[9b8@opendtn1m test_dir]$ rm test_file.txt
[9b8@opendtn1m test_dir]$ cd ..
[9b8@opendtn1m ~]$ ls
foundational_hpc_skills hands-on-with-summit mp2.py test_dir
[9b8@opendtn1m ~]$ mv test_dir/ test_dir_delete/
[9b8@opendtn1m ~]$ rmdir test_dir_delete/
[9b8@opendtn1m ~]$ ls
foundational_hpc_skills hands-on-with-summit mp2.py
[9b8@opendtn1m ~]$
```

“cat” and “less” In Action

A terminal window with a title bar showing the user '9b8' and the current directory. The terminal displays a series of commands and their outputs. The 'cat' command is used to view the contents of 'file1c', which contains text about playing the clarinet. The 'less' command is then used to view the same file, and the cursor is visible on the first line of the output.

```
9b8 — 9b8@opendtn1m:~/foundational_hpc_skills/intro_to_unix/challenge/dir1 — ssh 9b8@opendtn.ccs.ornl.gov — 80x24

[9b8@opendtn1m dir1]$ pwd
/ccsopen/home/9b8/foundational_hpc_skills/intro_to_unix/challenge/dir1
[9b8@opendtn1m dir1]$ ls
file1  file1a  file1b  file1c
[9b8@opendtn1m dir1]$ cat file1c
This is file "1c".
Here is the letter "c" in quotes.
Users that can play the clarinet:
Carl
Mandy
Otis
[9b8@opendtn1m dir1]$ less file1c
[9b8@opendtn1m dir1]$
```

Many uses of cat

Command	Explanation
<code>cat file1.txt</code>	Display contents of file
<code>cat file1.txt file2.txt</code>	Concatenate two text files and display the result in the terminal
<code>cat file1.txt file2.txt > newcombinedfile.txt</code>	Concatenate two text files and write them to a new file
<code>cat >newfile.txt</code>	Create a file called newfile.txt. Type the desired input and press CTRL+D to finish. The text will be in file newfile.txt.
<code>cat -n file1.txt file2.txt > newnumberedfile.txt</code>	Some implementations of cat, with option -n, can also number lines
<code>cat file1.txt > file2.txt</code>	Copy the contents of file1.txt into file2.txt
<code>cat file1.txt >> file2.txt</code>	Append the contents of file1.txt to file2.txt
<code>cat file1.txt file2.txt less</code>	Run the program "less" with the concatenation of file1 and file2 as its input

Searching Within Files – grep 1

- Sometimes you want to search for patterns/strings in a file. As with other commands, `grep` has many options. The `grep` command searches for “regular expressions”...strings that contain characters with special meaning
- Simple case: find lines with the string ‘user’ in file1
`grep “user” file1`
- More complex: show lines ending with ‘user’ in file1
`grep “user$” file1`
- ...or perhaps lines beginning with ‘user’
`grep “^user” file1`
- Search all files in a dir1 for the string ‘user’
`grep -r “user” dir1/`

“grep” In Action

```
9b8 — 9b8@opendtn1m:~/foundational_hpc_skills/intro_to_unix/challenge/dir1 — ssh 9b8@opendtn.ccs.ornl.gov — 80x24
Mandy
Otis
[9b8@opendtn1m dir1]$ grep "Carl" file1c
Carl
[9b8@opendtn1m dir1]$ grep "is" file1c
This is file "1c".
Here is the letter "c" in quotes.
Otis
[9b8@opendtn1m dir1]$ grep "is$" file1c
Otis
[9b8@opendtn1m dir1]$ grep "^is" file1c
[9b8@opendtn1m dir1]$ grep "is" *
file1:This is file "1".
file1a:This is file "1a".
file1a:Here is the letter "a" in quotes.
file1b:This is file "1b".
file1b:Here is the letter "b" in quotes.
file1c:This is file "1c".
file1c:Here is the letter "c" in quotes.
file1c:Otis
[9b8@opendtn1m dir1]$ grep -w "is" file1c
This is file "1c".
Here is the letter "c" in quotes.
[9b8@opendtn1m dir1]$
```

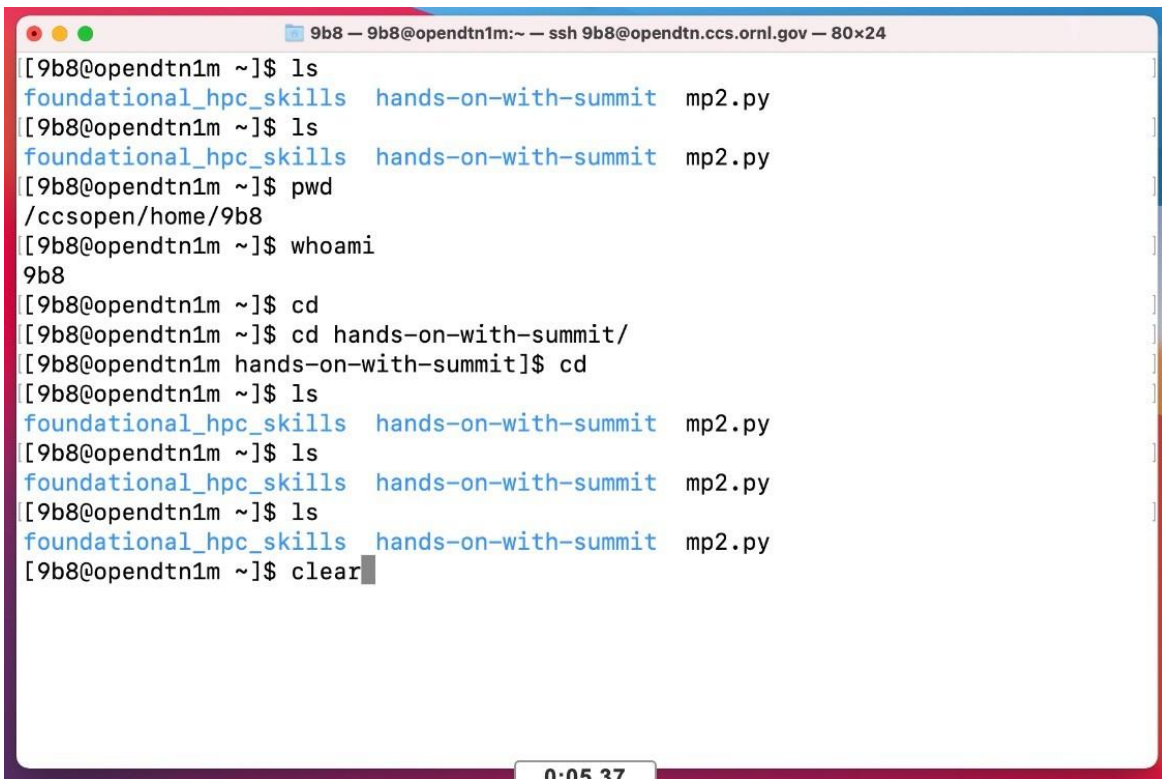

Searching Within Files – grep 2

- Normally, `grep` treat anything beginning with a hyphen as an option...even if it's in quotes
- The workaround is the `--` option, which tells `grep` that you're done giving options (and therefore any other hyphen is meant as an actual hyphen)
`grep -- "-2" file1`
- Grep does NOT recognize wildcards in the quoted string to search for. Use `"."` instead:

```
grep "user." file1
```

Bonus Tips 1

- “clear” is your friend:



```
9b8 — 9b8@opendtn1m:~ — ssh 9b8@opendtn.ccs.ornl.gov — 80x24
[9b8@opendtn1m ~]$ ls
foundational_hpc_skills  hands-on-with-summit  mp2.py
[9b8@opendtn1m ~]$ ls
foundational_hpc_skills  hands-on-with-summit  mp2.py
[9b8@opendtn1m ~]$ pwd
/ccsopen/home/9b8
[9b8@opendtn1m ~]$ whoami
9b8
[9b8@opendtn1m ~]$ cd
[9b8@opendtn1m ~]$ cd hands-on-with-summit/
[9b8@opendtn1m hands-on-with-summit]$ cd
[9b8@opendtn1m ~]$ ls
foundational_hpc_skills  hands-on-with-summit  mp2.py
[9b8@opendtn1m ~]$ ls
foundational_hpc_skills  hands-on-with-summit  mp2.py
[9b8@opendtn1m ~]$ ls
foundational_hpc_skills  hands-on-with-summit  mp2.py
[9b8@opendtn1m ~]$ clear
```

0:05.37

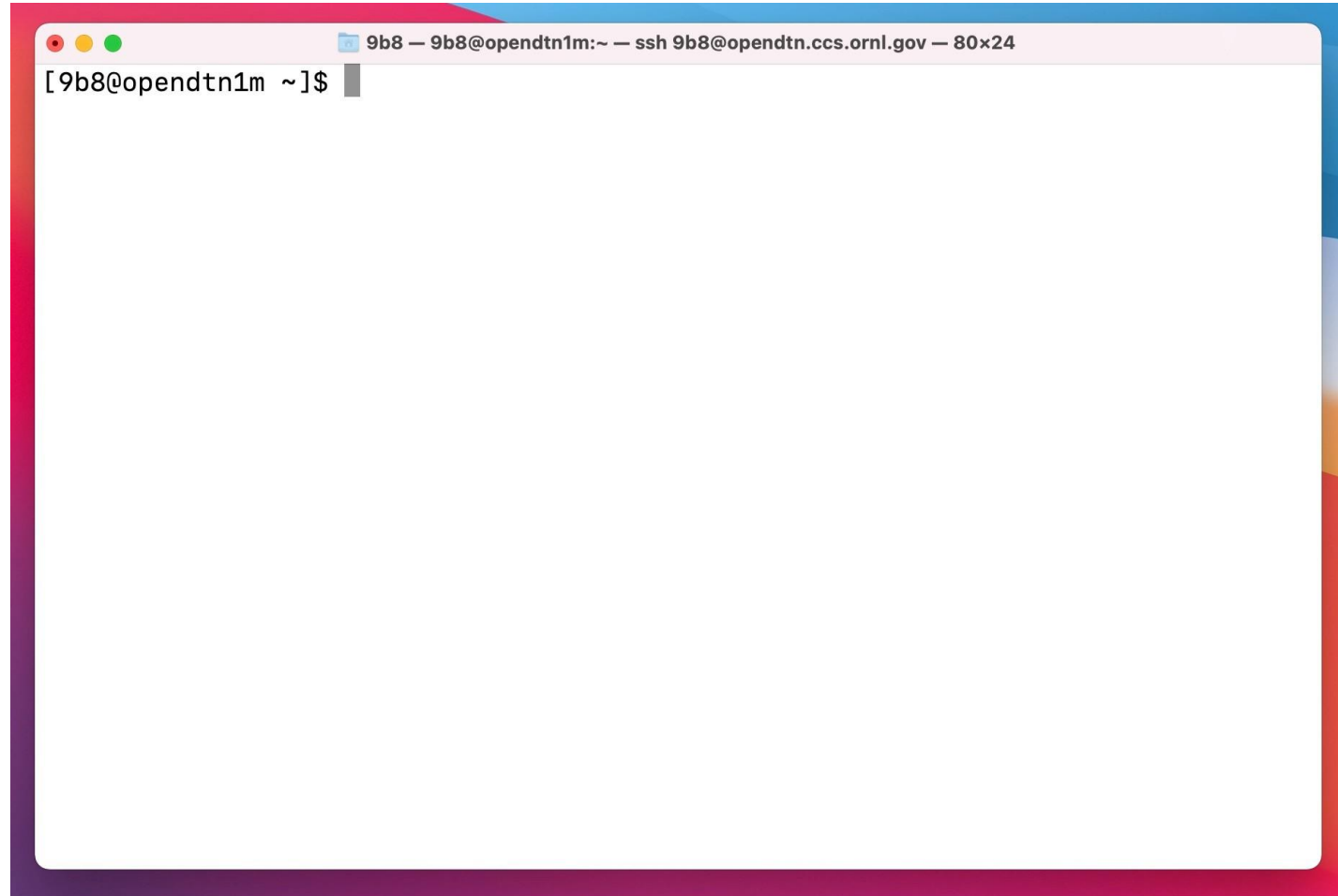


```
9b8 — 9b8@opendtn1m:~ — ssh 9b8@opendtn.ccs.ornl.gov — 80x24
[9b8@opendtn1m ~]$
```

0:07.16

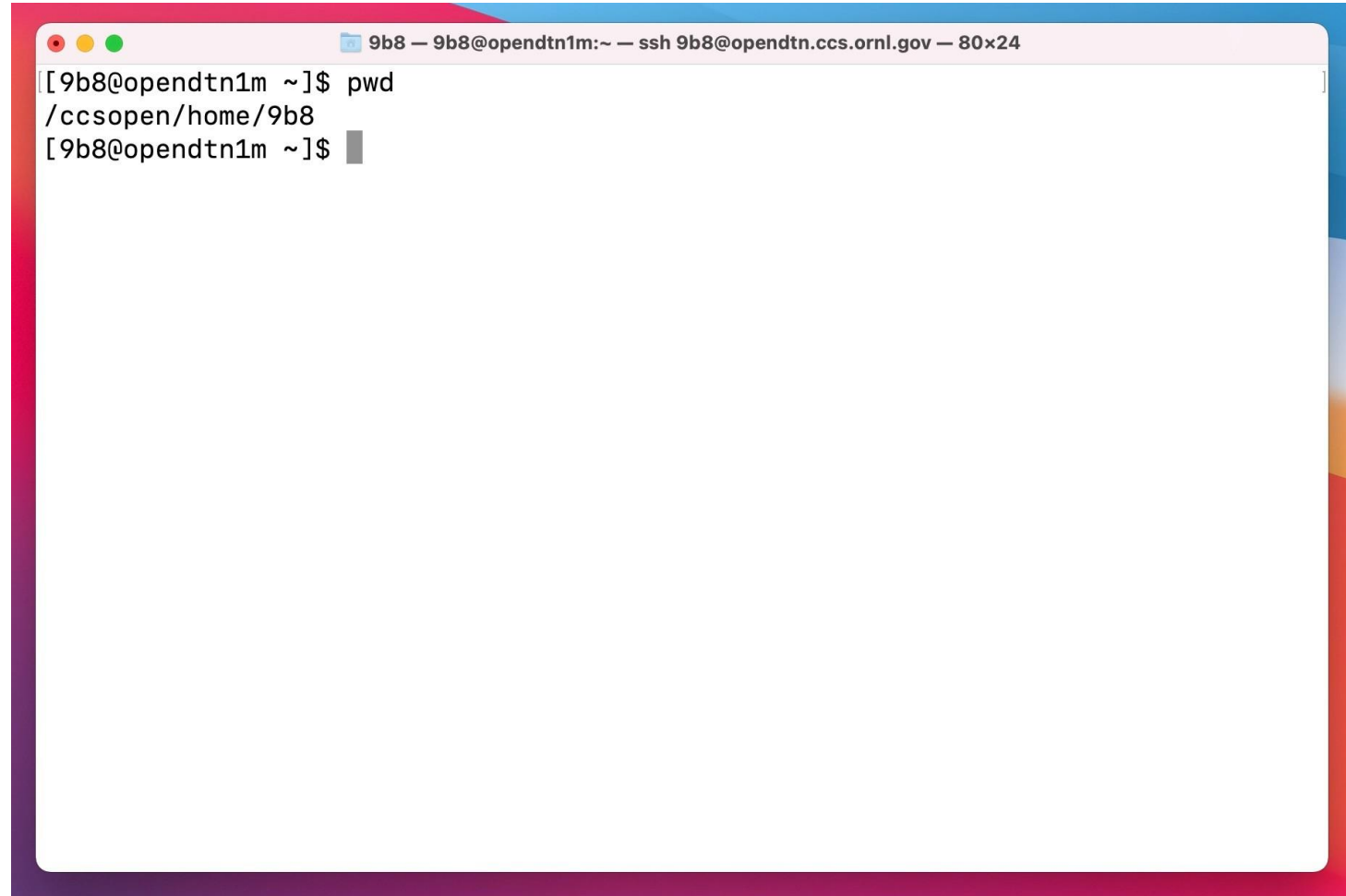
Bonus Tips 2

- Up/down arrows are also your friends:



Bonus Tips 3

- Tab-complete is your best friend:

A terminal window with a title bar that reads "9b8 — 9b8@opendtn1m:~ — ssh 9b8@opendtn.ccs.ornl.gov — 80x24". The terminal shows a user prompt "[9b8@opendtn1m ~]\$ " followed by the command "pwd". The output is "/ccsopen/home/9b8". The prompt is then shown again with a cursor, indicating the next step in the demonstration.

```
[9b8@opendtn1m ~]$ pwd
/ccsopen/home/9b8
[9b8@opendtn1m ~]$
```

Basic File/Directory Commands

Command	Description
pwd	P rint w orking (i.e. current) d irectory
man	Display the M anual for a command
whoami	Display the current user's username
mkdir	Create a directory (M a K e D IRectory)
rmdir	Delete a directory (R e M ove D IRectory)
cd	C hange (into a) d irectory
ls	L ist files
cp	C opy a file
mv	M ove a file (also used to rename a file)
rm	R emove (delete) a file
cat	Display the contents (con c atenate) a (hopefully text) file
less	Display the contents of a text file in a viewing mode
find	F ind files by filename
grep	Search for text within files
diff	Identify d ifferences in the contents of files

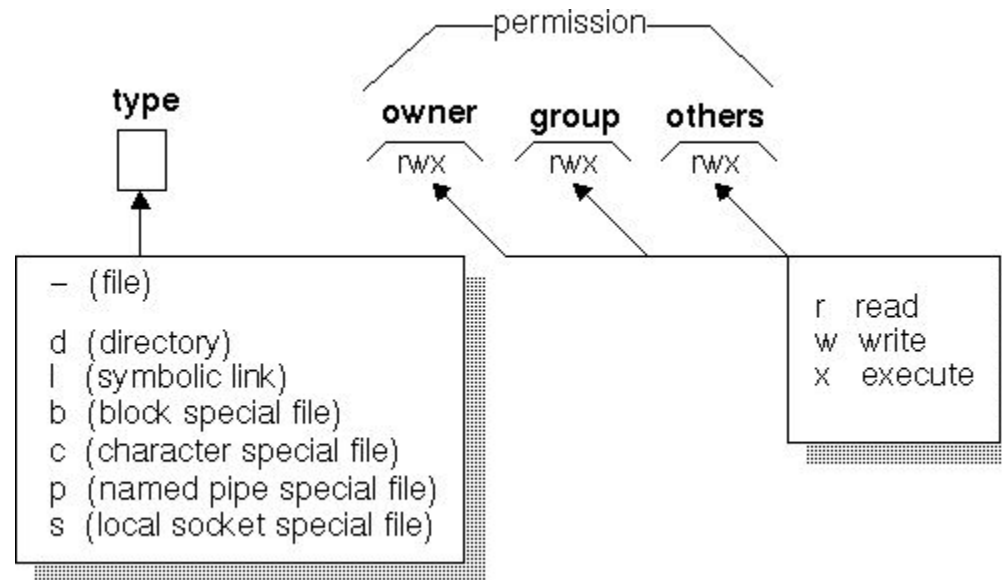
Extra Slides

Files

- Files are the basic entity for storing data
- Might contain a program, data, configuration info, an image, etc.
- Files have several attributes
 - Permissions: who can do what to/with a file
 - Owner: whose file is this
 - Group: to which group does this file belong

Permissions

- In the permissions string, the characters r, w, and x mean read, write, and execute permission is granted
- A - means the permission is not granted
- The permission groups always show read, write, execute in that order



Searching for files

- The `find` command lets you search for files on a huge variety of criteria
- It can also run commands on those files; this makes it one of the most powerful commands available
- General Syntax: `find [path/paths] [expression]`

```
$ find . -name "*data*" -print
```

```
$ find . -newer some_file
```

```
$ find /home/user1 ! -user user1
```

```
$ find . -group users -exec chgrp staff {} \;
```